

Database Design Document

Submitted By:

- **Ahd Montaser Jamea**

Abstract:

This project presents a database design for a ride-hailing system that helps users choose the best company based on trip price and estimated time.

The system allows users to compare different companies using trip details such as cost and duration.

The database was designed according to software engineering principles.

Python was used only to generate fake data and create static SQL files for testing purposes.

The database serves as a data storage layer accessed through backend APIs.

CHAPTER 1

INTRODUCTION

This chapter gives an overview of the aim, objectives, background and operation environment of the system:

PROJECT AIMS AND OBJECTIVES:

1. Design a structured database for ride-hailing services.
2. Store trip price and estimated time data.
3. Support comparison between different companies.
4. Ensure data accuracy and consistency.
5. Improve user decision-making.
6. Support future integration with backend APIs

Detailed Database Design:

1.User

- Primary key: user_id
-

Attributes:

- Name
- Email
- Password
- Phone
-
-

3.Drivers

- Primary key: driver_id
-

Attributes:

- Driver_name
- Car_model
- Phone

2.Company

Primary key: company_id

Attributes:

- Company_name
-
-

4.Trips

- Primary key: trip_id
-

Attributes:

- Pickup
- Dropoff
- Distance_km
- Time_min
- Price
-

In a well-organized **GoCheap** database, every table has a unique way to identify its records using **primary keys** and establishes relationships between tables through **foreign keys**.

The **User** table stores all registered users, uniquely identified by `<user_id>` as the primary key. Each user can make bookings or transactions, so `<user_id>` acts as a foreign key in the **Trips** table to link actions to the correct user.

The **Company** table stores all transport companies, uniquely identified by `<company_id>` as the primary key. Each company can have multiple drivers and trips, so `<company_id>` acts as a foreign key in the `*Driver*` and `*Trip*` tables.

The **Driver** table stores all drivers working for companies, uniquely identified by `<driver_id>` as the primary key. Each driver belongs to a company, so `<company_id>` acts as a foreign key linking drivers to their company in the **Company** table.

The **Trip** table stores all available trips, uniquely identified by `<trip_id>` as the primary key. Each trip is offered by a specific company and assigned to a specific driver, so `<company_id>` and `<driver_id>` act as foreign keys referencing the **Company** and **Driver** tables.

Relationships:

1. Trip ↔ User

Relationship: Request

Type: One-to-Many (1:M)

Description: A user can request multiple trips, but each trip is requested by only one user.

2. Trip ↔ Driver

Relationship: Drives

Type: One-to-Many (1:M)

Description: A driver can drive multiple trips, but each trip is driven by only one driver.

3. Company ↔ Driver

Relationship: Work_in

Type: One-to-Many (1:M)

Description: A company can employ multiple drivers, but each driver works in only one company.

4. Company ↔ Trip

Relationship: Owns

Type: One-to-Many (1:M)

Description: A company can own multiple trips, but each trip belongs to only one company.

1. BACKGROUND OF PROJECT:

PROCESSOR	<u>Intel(R) Core(TM) i7-6700HQ CPU @ 2.60GHz</u>
OPERATING SYSTEM	<u>Windows 10 Pro</u>
MEMORY	<u>32GB DDR5 8400MHZ</u>
HARD DISK SPACE	<u>0.5TB m.2 NVMe Gen6</u>
DATABASE	<u>MSQL Server</u>
Website of Diagrams	<u>Draw.io & Access</u>

CHAPTER 2

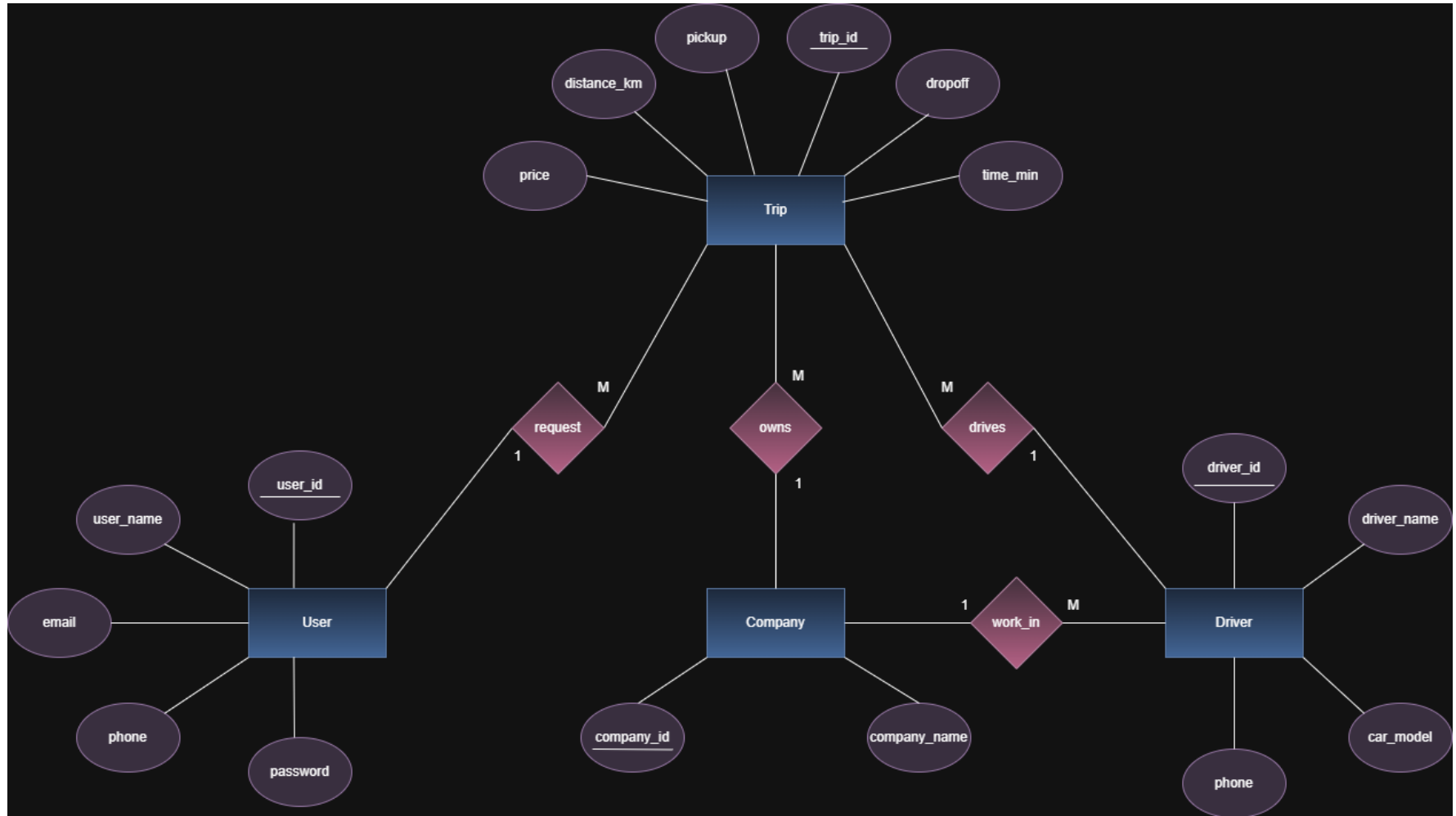
Database Design

1. Database Schema:

Figure 1: Schema diagram for GO Cheap:

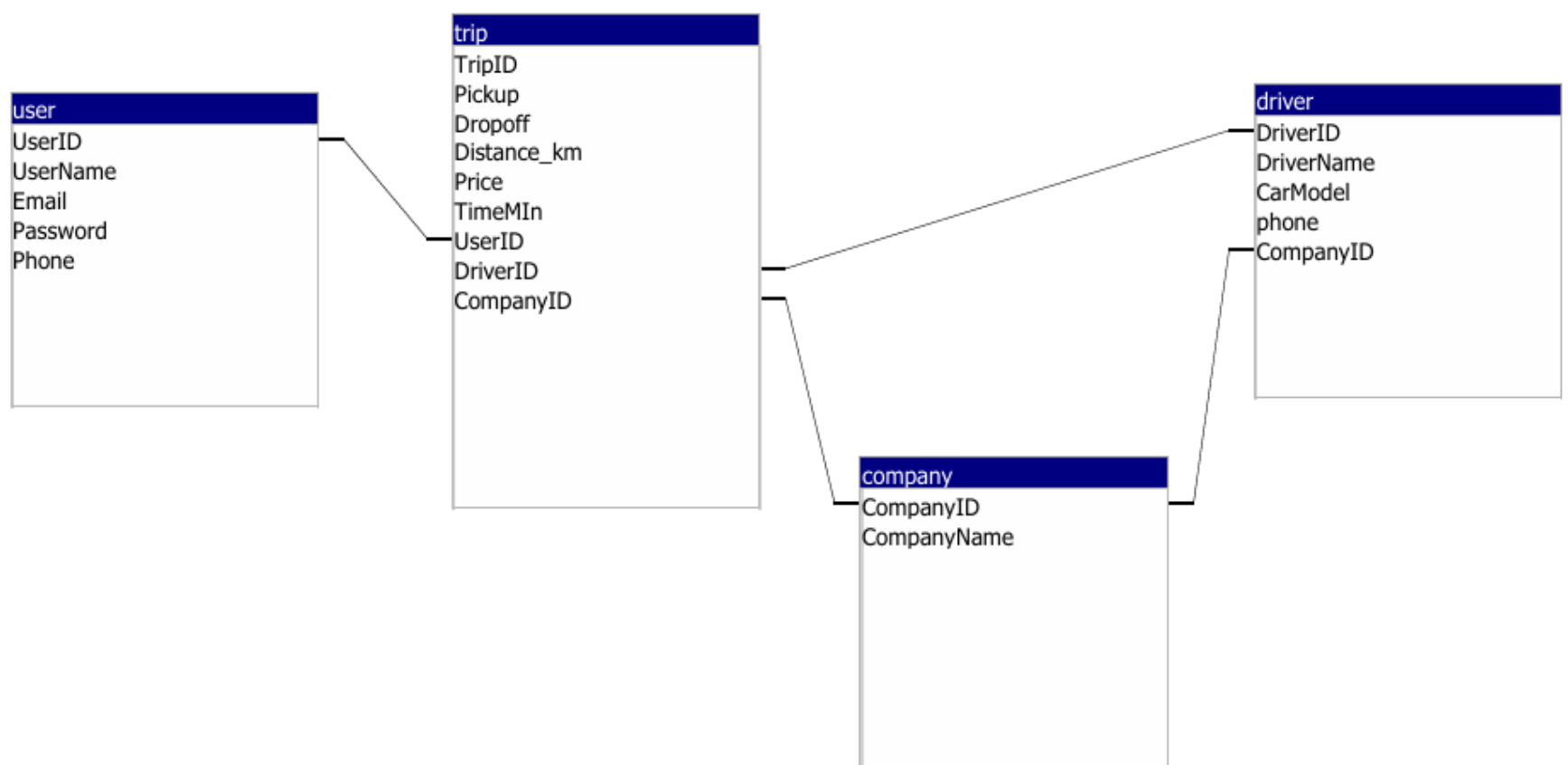


Figure 2: Relational diagram for GO Cheap:



2. TABLE DESIGN:

Relationships for GoCheap
Friday, December 19, 2025



CHAPTER 3

REFERENCES

- * *Uber* – <https://uber.com>
- * *InDrive* – <https://indrive.com>
- * *DiDi* – <https://web.didiglobal.com>
- * *ChatGPT* – <https://chat.openai.com>